1、语音房大型重构整理 —— 开发中解决过最难的问题是什么？

做过的最难的点是什么？线上出过的线上事故是怎样的？项目上做过哪些优化，期间遇到一些什么问题，是通过什么思路去解决的问题？

背景

- **可读性差**：本身业务模块多，比如有RTM消息收发、RTC消息收发、消息展示区、底部工具栏、麦位区、成员列表、房间交互区（点赞、申请上下麦）、房间信息、小游戏、礼物模块、运营活动模块等。所以各种业务逻辑的代码混合在一起。而且大部分在VC区，即使使用。ViewController 代码量 3000多行，另外有3个分类。累计大概四五千行的样子。早期采用MVVM的框架。修改一处就得了解全部代码，非常不友好；尤其是不熟悉该模块的同事做语音房需求，代码逻辑复杂，非常掣肘！
- **扩展性差**：新增、修改每个功能需要改动多个相关的类或布局，明明业务逻辑独立的功能，却需要大量耦合已有功能，修改已有代码，甚至引起连锁问题，修一个问题，结果又出了一个新问题。
- **维护人员多**：统计 commit 历史，多人提交代码，改动时相互的影响、冲突不断。

实践步骤：

- 1、梳理业务
- 2、架构选择与设计
- 3、逐步替换
- 4、验证上线

架构设计

框架选型：

什么样的框架可以让模块间业务代码相对更加独立？同时满足可读性、扩展性、可维护性等问题。而且还需要考虑的点是各个模块都会共用部分公共数据和操作。于是就希望有个类似树状的一个框架雏形：可以让多条业务线各自相互不影响，同时可以共用公共部分核心数据。而且最重要的，各个业务视图都是与VC或多或少相关联的。

好的架构应该关注的几个点：

- **代码均摊**：合理的App架构应该合理分配代码。每个类、结构体、方法甚至变量，都应严格遵守**单一职责**的设计原则。我们常用的**MVC、MVVM、MVP**等框架都是为了让代码更加合理。

常用的架构框架

- **便于测试**：主要是为了覆盖测试用例，提高代码质量。
- **易用性**：主要是值代码的易扩展性、可维护性、适合多人协作等。

最后，确定的方案是使用**插件化为主，插件内MVVM的架构**：

模块插件化，插件内MVVM

- 将所有的大小模块抽象成插件。每个插件相互独立。然后通过不同的插件组合，来适应不同的语音房业务。比如：主题房的麦位与普通语音房的不一样、高级语音房的礼物与普通语音房不一样。
- 插件内，是一个独立可插拔的模块，比如麦位插件，插件里持有当前麦位的UI、麦位的ViewModel（ViewModel负责对麦位数据进行校验和刷新、处理上下麦的请求与消息发送等等。）、麦位的Model等。

Q：插件如何同步VC的生命周期？

PageLife

- 插件与主VC的联系，是使用“**PageLife**”框架管理进行管理。PageLife主要是通过观察者模式，将主VC的一条生命线扩展成多条生命线。主VC只负责分发。 然后让插件与主VC同步，在对应的生命周期函数里执行对应的方法。

> 具体实现就是：
>
> 1、声明一个声明周期的协议**TPPageLifeService**，协议里的方法与VC生命周期函数同步。
>
> 2、每个插件都实现这个协议，这样单个插件的生命周期与VC的生命周期就是一致的。
>
> 3、因为所有插件都是独立的，所以需要使用一个**PageLifeManager**对插件进行管理：manager里持有一个**Set**，存放当前业务注册的插件。
>
> 4、主VC持有Manager，当执行到某一个生命周期函数时，通过manager对其持有的插件集合进行遍历，让每一个插件都进行对应生命周期协议函数的执行。
>
> Q：如何设计一个各个模块共用的基础数据？而且要确保数据的唯一性！如何保证所有数据不会随意被任意插件该乱？
>
> 所以就需要一个统一上下文。
>
> 5、每个插件可能共用一套基础的数据，比如语音房Model，所以需要添置一个**上下文协议Context**。这个协议里定制每个插件需要从主VC获取的公共属性，比如VC的View，基础info。所以上下文协议由主VC实现。

> 6、有了以上内容后，那单个插件的视图就可以在ViewDidLoad时，通过context上下文拿到主VC的视图，然后让其添加进去。当然这种插件视图只能相对于主视图进行布局
>
> Q：非UI的逻辑模块是否可以单独做成插件？
>
> 7、当然了，并不一定所有的插件都是因视图进行拆分的，也有纯功能型的插件，比如消息发送插件、底层SDK插件等。也有只用于适配的容器型插件，比如底部tools容器，则主要是用于适配。
>
> 9、有的插件可能需要在视图生命周期函数之前做一些事情，比如在ViewDidLoad之前注入自己的对外接口。所以可以在VC的生命周期函数之上，**扩展插件的生命周期**
>
> 10、第7点说到的插件都是相对于主VC进行布局，但是插件视图之间有可能是同层级或add的先后顺序问题。所以又需要**扩展插件的优先级**，确保优先级高的插件优先执行生命周期函数。

Q：插件之间如何通讯？

插件间通讯：Swinject

- **插件之间肯定也是需要进行通讯的**，比如麦位操作后，将事件通过RTM消息同步给其他人，让其他人去刷新麦位。这就涉及到麦位插件与消息插件之间如何进行通讯。这里是参照依赖注入框架**Swinject**简单实现的：

> 1、首先插件使用接口进行对外通信。（插件实现接口协议，外部使用协议接口对插件发送消息）
>
> 2、声明一个**协议TPContainerProtocol**，协议实现对**插件对外接口**进行管理。这里主要是通过字典对协议类型进行持有，然后提供注册和查询的接口

代码块

```
public protocol TPContainerProtocol: AnyObject {
    var services: [ServiceKey: Any] { get set }
}

extension TPContainerProtocol {
    // 注册
    public func register<T>(_ serviceType: T.Type, name: String? = nil, factory: @escaping (Self) -> T?) {
        let key = ServiceKey(serviceType: serviceType, argumentsType: NoArgumentsFunctionType.self, name: name, option: nil)
        services[key] = factory
    }
    // 查询
    public func resolve<T>(_ serviceType: T.Type, name: String? = nil) -> T? {
        let item = services.filter { item in
```

```
            return item.key.serviceType == serviceType && name == item.key.name
        }.first
        guard let item = item else { return nil }
        if item.key.argumentsType == NoArgumentsOjectType.self {
            return item.value as? T
        } else if item.key.argumentsType == NoArgumentsFunctionType.self {
            let facotry = item.value as? ((Self) -> T?)
            return facotry?(self)
        } else {
            return nil
        }
    }
}

```

3、这里要注意的一点是，如果对外接口是一个协议，如何对协议进行注册和查找？所以需要**将协议进行包装成一个Struct, 让其作为Key**，更方便快捷进行查找：

代码块

```
public protocol ServiceKeyOption: CustomStringConvertible {
    func isEqualTo(_ another: ServiceKeyOption) -> Bool

    func hash(into: inout Hasher)
}

// MARK: - ServiceKey

public struct ServiceKey {
    let serviceType: Any.Type
    let argumentsType: Any.Type
    let name: String?
    let option: ServiceKeyOption? // Used for SwinjectStoryboard or other extensions.
}
```

4、这样所有插件的所有对外接口就都能够注册和查询到。所以使用一个TPContainer类，实现这个协议，然后使PageLifeManager持有这个TPContainer，这样就可以通过Manager获取到当前所有的插件的所有对外接口。

代码块

```
public class TPContainer: TPContainerProtocol {
    public var services = [ServiceKey: Any]()
```

```
}
```

5、具体使用：

代码块

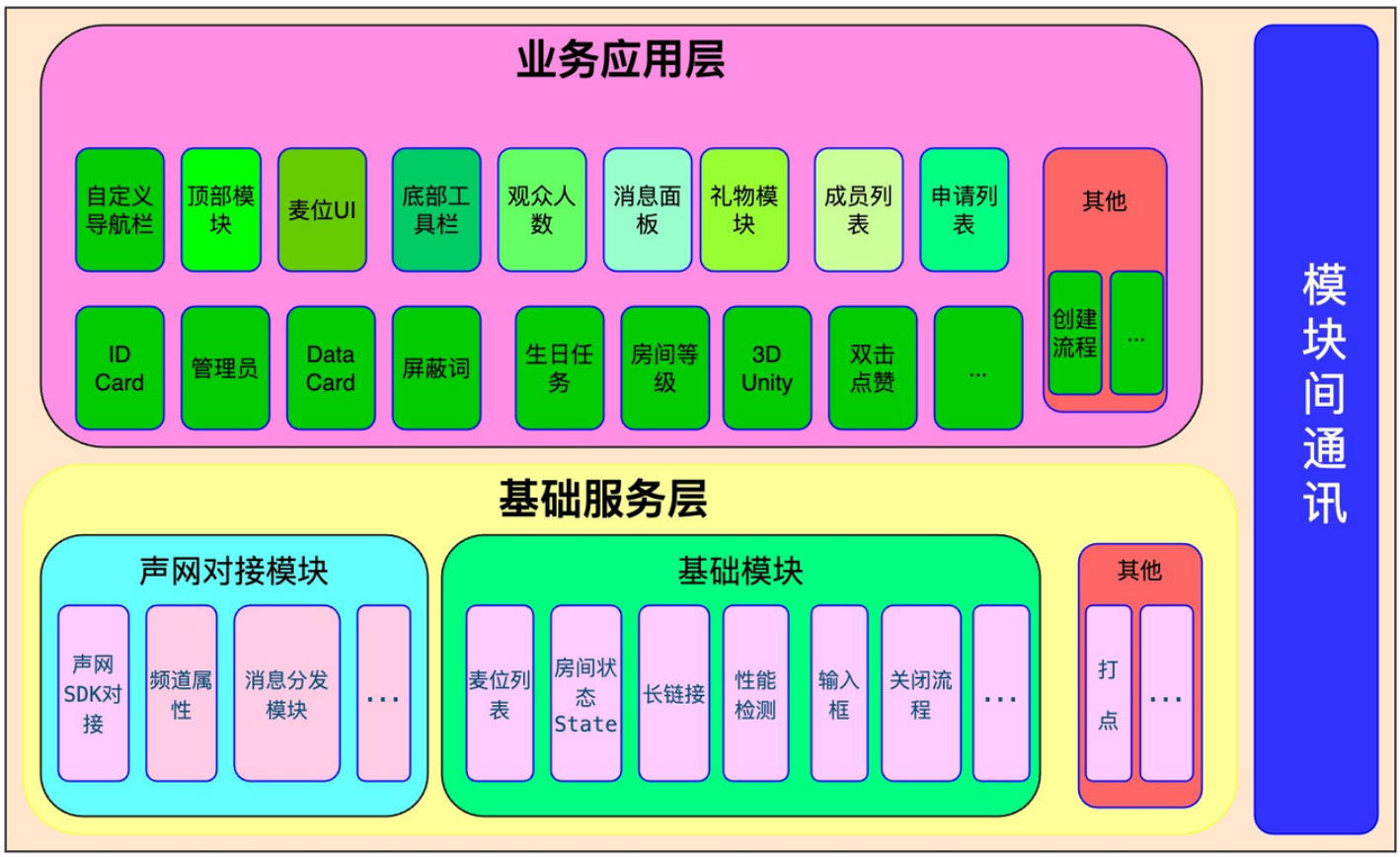
```
// 声明对外接口
protocol TPTestInterface {
    /// 收到消息
    func receiveMessgae()
}

// 注册接口

class XXPlugin: BasePlugin, TPGiftAnimateSerciveInterface {
    func pluginDidLoad() {
        self.context?.container.register(TPTestInterface.self) { [weak self] _ in
            guard let self = self else { return nil }
            return self as? TPTestInterface
        }
    }

    /// 收到消息
    func receiveMessgae() { xxx }
}

// 查询接口
class YYPlugin:  BasePlugin {
    func sendMessage() {
        if let service = self.context?.container?.resolve(of: TPTestInterface.self) {
                // 发送消息
                 service.receiveMessgae()
        }
    }
}
```

Q：适配的层级关系，先后顺序怎么处理？

项目分层

主要是为了将部分共用的插件固定下来，减少代码架构的劣化。其次像基础服务层，可能会服务于各种不同的类型的语音房，比如居酒屋、宇宙通话、主题房等。

所以语音房模块，主要是分了基础服务层、业务应用层：



- SDK层：主要是对接声网的SDK，包括RTM插件、RTC插件
- 基础服务层：频道的创建插件、房间基础信息、消息处理插件等
- 业务应用层：上层业务，比如礼物插件、身份卡片插件、消息显示插件等

消息体系

首先消息主要是指语音房的各种消息。首先语音房有频道消息、显示型消息、显示型消息又分为文本、图片、图文等。

消息结构：

设计一个合理的消息Model，有助于消息的扩展和消息的统一处理。

所以整个消息分为了3大类：频道消息、UI消息和非UI消息：

- UI消息：顾名思义就是需要显示在屏幕上的消息。通常显示型消息，会有用户的一些装扮信息，比如气泡、称号等。
- 非UI消息：主要是指一些广播消息，比如麦位变更消息
- 频道消息：就是有平台发出的消息，比如服务端推过来的一条消息、房间公告等消息

> 1、之所以要将非UI型消息和频道消息区分开来，主要是发送者的不同，频道消息通常是平台进行发送的，没有固定发送者，比如语音房内公告的变更、有人玩游戏获奖了等等。这类消息一半到了接收方，大部分需要转化成显示消息
>
> 2、非UI消息就是指有固定发送消息的人的，比如房主操作了上下麦、有人申请了上下麦等等。成员之间的相互关注等。
>
> 3、

所以整个Model是设计成两层结构体

代码块

```
struct AudioLiveMessage: TPJson {
    var messageType: AudioLiveMessageType // 3种类型
    var content: Any? // 对应具体的消息结构
    var ver: Int?
 }

 struct UIMessage: TPJson {
     var sender: User
     var type: UIMessageType
     var content: Any?
 }

```

消息分发

在语音房中，通常一个消息会引起多个模块的响应，比如如果有人进房间的消息，成员列表插件需要更新数据、公屏上需要显示欢迎消息、进房的人本地还会插入点赞提醒、关注提醒等。

所以通常一个消息会对应多个响应者。

所以根据观察者模式，设计一个类似通知中心的管道，对消息进行分发：

- 需要一个消息处理插件，主要是为了接收到RTM消息时进行消息解析，然后将解析好的消息进行消息分类。然后再对消息进行派发
- 所以消息处理插件，需要持有一个**ObserverCenter**；需要监听消息的插件都注册到ObserverCenter里面，然后收到消息时，遍历Center，对每个注册了该消息的观察者派发消息。

1、ObserverCenter的实现：

  - 使用属性包装器，将观察者进行weak包装，包装好的对象弱引用持有注册对象；
  - 其次使用ObjectIdentifier生成identifier作为存储的key。
  - ObserverCenter维护一个字典，对注册的对象进行管理

代码块

```
protocol ObserverWrapper {
    var value: AnyObject? { get }
    func set(observer value: AnyObject)
}

class ObserverCenter {
    private class WeakObjectWrapper: ObserverWrapper {
        private(set) weak var value: AnyObject?
        func set(observer value: AnyObject) {
            self.value = value
        }
    }

    private var observerWrappers = [ObjectIdentifier: ObserverWrapper]()

    func add(observer: AnyObject) {
        let wrapper = WeakObjectWrapper()
        wrapper.set(observer: observer)
        let identifier = ObjectIdentifier(observer)
        observerWrappers[identifier] = wrapper
    }

    func remove(observer: AnyObject) {
        let identifier = ObjectIdentifier(observer)
        observerWrappers.removeValue(forKey: identifier)
    }

    func clear() {
        observerWrappers.removeAll()
    }

    var observers: [ObjectIdentifier: AnyObject] {
        return Dictionary(uniqueKeysWithValues: observerWrappers.map { (key, value) in
            return (key, value.value)
        }).compactMapValues { $0 }
    }

    func forEachObserver(_ block: (_ key: ObjectIdentifier, _ value: AnyObject) -> Void) {
        observers.forEach { item in
            block(item.key, item.value)
        }
    }
}
```

2、使用：

代码块

```
protocol MessageDelegate: AnyObjec {
    func getUIMessage(_ UIMessage xxx)
}

class MessagePlugin: xxx {
    private let msgCenter = ObserverCenter()
    // 这里是弱引用持有
    func add(observer: MessageDelegate) {
        msgCenter.add(observer: observer)
    }
}

extension MessagePlugin: RTMMessageDelegate {
    func getMessgae(_ message: AudioLiveMessage) {
        // 消息解析
        msgCenter.forEachObserver { _, observer in
            if let observer = observer as? MessageDelegate {
                observer.getUIMessage(xxx)
            }
        }
    }
}
```

- 设置消息转发插件，有一个好处就是能够统一处理消息，对数据打点、分析、消息派发、频率控制，都很好处理。还可以在这个插件里处理一些有相互依赖关系的消息、处理消息的优先级等等。

消息显示

显示型的消息，最终都是需要显示在公屏上的。所以显示模块是最复杂，也最影响性能的地方。

消息显示映射

首先，我们发现虽然是不同的消息类型，但是可能对应相同的显示样式；所以消息样式的类型Cell类型和消息类型不能是一一对应的关系，应该是一对多的关系。

但是相同的消息样式（cell样式），它所显示的内容和交互又是和消息类型息息相关的。

其次，这些样式，后续也用到了其他业务场景。所以设计一套扩展性强、可公用的消息样式是非常有必要的。

这里使用**策略模式**，对语音房的显示消息进行映射

- 设计一个两个协议，用于绑定显示类型与显示样式

代码块

```
protocol MessageUIItemProtocol {
    var showType: MessageShowType { get }
    var cellClassType: MessageCellInterface.Type { get }
}

/// 每一种消息类型Cell都应该实现的协议
protocol MessageCellInterface where Self: UITableViewCell {
    static var identifier: String { get }

    /// 设置model
    func setLiveMessage(_ message: TPMessageUIModel, uiItem: MessageUIItemProtocol)
}

extension MessageCellInterface {
    static var identifier: String {
        return Self.description()
    }
}

enum MessageShowType: CaseIterable {
    case none   // 提示版本升级? 无法解析?
    case showTextMsgUI   // 展示文本消息
    case showHintMsgUI   // 提示消息
    case showImageMsgUI // 图片消息
    ...
}

```

- 消息显示Model与Cell类型绑定

代码块

```
class TPMessageUIModel {
    var content: Any?
    var type: MessageShowType
}
```

- 不同的消息类型对应不同的UIItem，主要是为了处理事件和内容

代码块

```
/// 提示类型消息
struct TPLiveHintMessageUIItem: TPLiveMessageUIItemProtocol {
    var showType: TPAudioLiveMessagePanelShowType {
        return .showHintMsgUI
```

```
    }

    var cellClassType: TPAudioLiveMessageCellInterface.Type {
        return HintMessageCell.self
    }

    private var message: HintMessage?
    private weak var delegate: MessageDelegate?
    init(delegate: MessageDelegate?) {
        self.delegate = delegate
    }

    func hintMessage(from text: Any) -> String? {
        // 解析model消息
        message = HintMessage(from: text)
        return message.hintText
    }

    func dicClickedAction() {
        // 交互回调出去
        delegate?.xxxx
    }
}
```

- Cell里的数据通过UIItem获取

代码块

```
class HintMessageCell: MessageCellInterface {
    ... ...
    private var delegate: TPLiveHintMessageUIItem? {
        return uiItem as? TPLiveHintMessageUIItem
    }

    func setLiveMessage(_ message: TPMessageUIModel, uiItem: TPLiveMessageUIItemProtocol) {
        let text = delegate?.hintMessage(from: message.text)
        hintLabel.text = text
    }

    @objc func clickedHintBtn() {
        delegate?.dicClickedAction()
    }
}
```

这样，样式Cell就与Model完全解耦了。UIItem与实际消息model对应

- UITableView中注册

代码块

```
class MessagePannel {
    // 将每一种
    var messageUIItems: [MessageUIItemProtocol] {
        return [TPLiveHintMessageUIItem(delegate: self)]
    }
    ...
     private(set) lazy var tableView: UITableView = {
        let tableView = UITableView()
        ... ...
        for messageCell in messageUIItems {
            tableView.register(messageCell.cellClassType, forCellReuseIdentifier: messageCell.cellClassType.identifier)
        }
         return tableView
     }

     ... ...
     func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) -> UITableViewCell {
        guard var model = dataSource[safeIndex: indexPath.row],
              let uiItem = messageUIItems.first(where: { $0.showType == model.type }),
              let cell = tableView.dequeueReusableCell(withIdentifier: uiItem.cellClassType.identifier, for: indexPath) as? MessageCellInterface else {
            return UITableViewCell()
        }
        if model.type == .none {
            model.text = TPL10n.chatRoomLowVersionHint // 无法识别的消息类型
        }
        cell.setLiveMessage(model, uiItem: uiItem)
       return cell
    }
}
```

消息刷新

在进行测试时发现，公屏消息刷新速度与卡顿成正比。

所以需要对消息刷新策略进行调整。

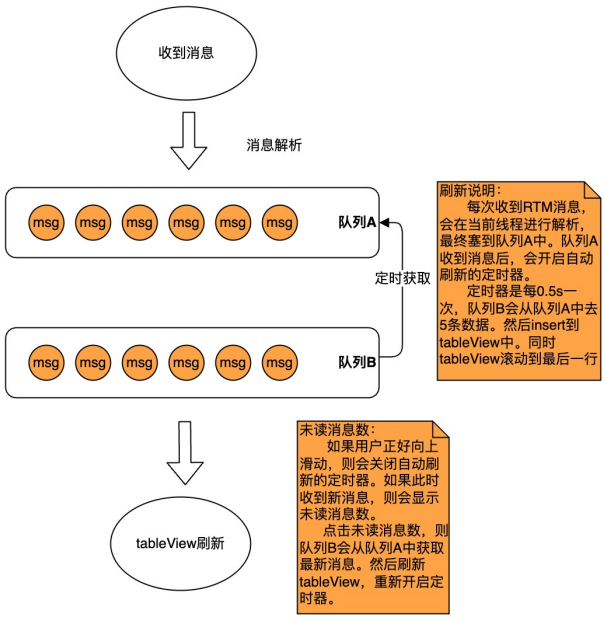
双队列定时刷新方案

- 队列A，可以持续对消息进行接收，并且在接收消息的同时先进行消息解析或映射；然后入队时，触发一个自动刷新的定时器；

> 注意，这里队列A使用的是一个多读单写的队列，以保证线程安全。它是使用dispatchBarrier进行的
>
> - 自定义一个并发队列（Barrier需要在自定义的并发队列才有效）
> - barrier写，sync读。

- 队列B，用于存放将要显示到公屏上的消息。
- 定时器每一个时间阈值内（假设0.5s），队列B会从队列A中读n条消息，然后触发tableView刷新。

> 实际操作中，队列B就是TableView的显示数组。
>
> 这里要注意的是：去取的时候，当前是在主线程的，而队列A，如果正在写，那么会阻塞读取数据的操作
>
> 所以读的时候，可以使用一个串型队列，异步读取



代码块

```
class TPMessageDataManager {
    fileprivate let msgReadQueue = DispatchQueue(label: "com.chat.chatMessageRead")
    private var storageDataArr = SafeArray<MessageModel>()
    private var timer: Timer? // 防抖
    weak var delegate: TPAudioLiveMessageDataManagerDelegate?

     private func starTimer() {
        guard timer == nil else { return }
        stopTimer()
```

```
        timer = Timer.every(TimeInterval(0.5), mode: .common, execute: { [weak self] _ in
            guard let self = self else { return }
            // 提醒UITableView来读取数据
            self.delegate?.liveMessageDataManagerHeartbeat()
        })
    }

    /// 收到新消息，这里本身就是在后台线程中处理的。
    func receiveNewMessage(_ msg: MessageModel) {
        // 可以在这里对model的适配数据进行计算和临时缓存
        self.storageDataArr.append(message)
        self.starTimer()
    }

    /// UITableView从缓存数据中读取数据到dataSource
    func readMsgFromStorage(_ count: Int, _ completion: @escaping (_ message: [TPAudioLiveMessageModel]) -> Void) {
        self.msgReadQueue.async { [weak self] in
            guard let self = self else { return }
            let msg = self.storageDataArr.removeAndReturnPreElements(count)
            completion(msg)
        }
    }
}
```

性能优化

- 上面的双队列定时刷新策略
- 然后读取的时候限制一次性显示数据的大小，控制刷新时间间隔；
- 提前计算cell中显示的数据，比如文本高度、富文本生成等；
- 公屏幕上的消息数达到一定程度后，及时清理；
- 对图片进行处理，离屏渲染、光栅化缓存、缩略图、单独使用缓存，退出房间后及时清理；
- 减少接口请求频次和UI刷新频次：比如有人员进出，列表不立马进行刷新，一定时间间隔内刷新，大语音房达到一定人数后，弱化实时性；
- 控制一瞬间消息爆炸问题

iOS性能优化实践01：卡顿

iOS性能优化实践02：内存

重构质量保证

- monkey测试
- AB测试

抖音iOS最复杂功能的重构之路--播放器交互区重构实践

iOS依赖注入：技术原理+应用场景，四种方式+源码解读

iOS插件化